LAHORE GARRISON UNIVERSITY

Department of Computer Science

PROJECT DOCUMENTATION

Mini Programming Language

Roman Urdu Compiler & Web IDE

Project Title	Mini Programming Language - Roman Urdu Compiler & Web IDE
Semester	6th Semester
Course	Compiler Construction Lab
Instructor	Sir Azib Mahmood
Date	May 2026

Team Members

#	Name & Roll No.
1	M. Fezan (Fall-23-BSCS-466)
2	Shoaib Rafiq (Fall-23-BSCS-479)
3	Umer Malik (Fall-23-BSCS-628)

1. Background of the Project

Programming languages like Python, Java, and C++ use English keywords (*if*, *while*, *print*, *return*) that create a barrier for non-English speakers. In Pakistan and other South Asian countries, millions of people communicate daily in Urdu but have limited English proficiency. This creates an unnecessary obstacle to learning programming.

The concept of non-English programming languages has existed since the 1980s - languages like Hindi Programming Language (HPLC), Bengali Programming Language (BASIC Bangla), and Arabic-based languages like Jarb have explored this space. However, most of these efforts produced simple translators rather than full compiler implementations.

This project takes a different approach. Rather than building a thin wrapper around an existing language, we designed and implemented a **complete compiler pipeline** for a custom programming language with **Roman Urdu** (Urdu written in English letters) syntax. Roman Urdu was chosen because it is the most natural way Urdu speakers type on standard keyboards - it is how billions of Urdu messages are typed daily on WhatsApp, Facebook, and other platforms.

The result is a Mini Programming Language that supports variables, arithmetic, control flow (if/else, while loops), functions with recursion, arrays, user input, type casting, block scoping, and a full optimization pass - all written in Urdu keywords.

Motivation

- Remove the English-language barrier for Pakistani students learning to program for the first time
- Demonstrate that a full compiler pipeline - lexing, parsing, semantic analysis, IR generation, optimization, codegeneration, and interpretation - can be built entirely from scratch in Python without external tools like PLY or
ANTLR
- Serve as a comprehensive semester project for the Compiler Construction Lab course, covering every topictaught in the curriculum with a working, deployable implementation
- Provide an educational platform where every intermediate compiler artifact (token stream, AST, symbol table, TAC, optimized TAC, and generated Python code) is visible to the user

2. Introduction

Mini Programming Language is a custom programming language with Roman Urdu syntax, paired with a complete compiler and interpreter implementation. The project consists of two main components:

1. **Backend** - A Python-based compiler that implements a full 7-stage pipeline: Lexical Analysis, Parsing, Semantic Analysis, IR Generation, Optimization, Python Code Generation, and Tree-Walk Interpretation. Served through a FastAPI REST API on port 8008.
2. **Frontend** - A React-based Web IDE featuring the Monaco Editor (the same editor that powers VS Code) with custom Urdu language syntax highlighting, real-time error marking, and tabbed visualization of every compiler stage. Runs on port 5173 via Vite.

A user writes code in Roman Urdu using keywords like **rakho** (declare), **dikhao** (print), **agar** (if), **warna** (else), **jabtak** (while), **banao** (function), **wapis** (return), and **karo** (call). When they press "Chalao" (Run), the code travels through all 7 pipeline stages, and the results - tokens, AST, semantic analysis, optimized TAC, generated Python, and program output - are displayed in real time.

Key Design Principles

- **Zero external compiler tools** - No PLY, ANTLR, or parser generators. Every stage is hand-written from scratch in pure Python

- **Educational focus** - Every pipeline stage is visible to the user. The IDE is designed to help students understand how compilers work
- **Urdu-first error messages** - All error messages in Roman Urdu with "did you mean?" suggestions for typos using Levenshtein edit distance
- **Block scoping** - Variables declared inside agar (if) and jabtak (while) blocks are local to that scope
- **Safety hardened** - 10,000 iteration cap and 100-level call-depth limit prevent infinite loops and stack overflows

Project Scope

The project is scoped as a semester deliverable demonstrating compiler theory in practice. It is not intended to be a production-grade general-purpose language but rather a fully correct, educationally complete implementation covering all major compiler construction concepts.

3. Project Details

3.1 Language Reference

Keywords

The language uses **16 keywords**, all written in Roman Urdu:

Keyword	English Meaning	Usage
rakho	keep/put	Variable declaration: rakho x = 10
dikhao	show	Print to screen: dikhao "hello"
agar	if	Conditional: agar x > 5 ... khatam
warna	else	Else branch within agar block
jabtak	until/while	While loop: jabtak x > 0 ... khatam
khatam	end/finished	Block terminator for agar, jabtak, banao
sahi	correct/true	Boolean literal true
ghalat	wrong/false	Boolean literal false
aur	and	Logical AND operator
ya	or	Logical OR operator
banao	make/build	Function definition: banao f(p) ... khatam
wapis	return	Return from function: wapis expr
functionbnao	make function	Function definition
wapisbejo	send back	Return from function

Operators

Operator	Meaning
+ - * /	Arithmetic
> < >= <= == !=	Comparison
!	Logical NOT
aur	Logical AND
ya	Logical OR
=	Assignment

Data Types

- **Numbers** - Integers and floats: 42, 3.14
- **Strings** - Double-quoted: "Assalam o Alaikum"
- **Booleans** - sahi (true), ghalat (false)
- **Arrays** - Ordered lists: [1, 2, 3], ["a", "b", "c"]

Syntax Examples

Variables & Printing:

```
rakho naam = "Duniya"
dikaho naam
```

Conditionals:

```
rakho x = 10

agar x > 5

dikaho "x bara hai"

warna dikaho "xc
hota hai"

khatam
```

While Loop:

```
rakho x = 3
jabtak x > 0
dikaho x rakho
x = x - 1
khatam
```

Functions:

```
functionbnao add(a, b)
wapis a + b

khatam

rakho result = add(10, 20)
dikaho result
```

Arrays:

```
rakho fruits = ["Aam", "Seb", "Kela"]
dikaho fruits[0]
```

3.2 Architecture & System Design

The project follows a monorepo structure with two loosely coupled subsystems communicating over a REST API:

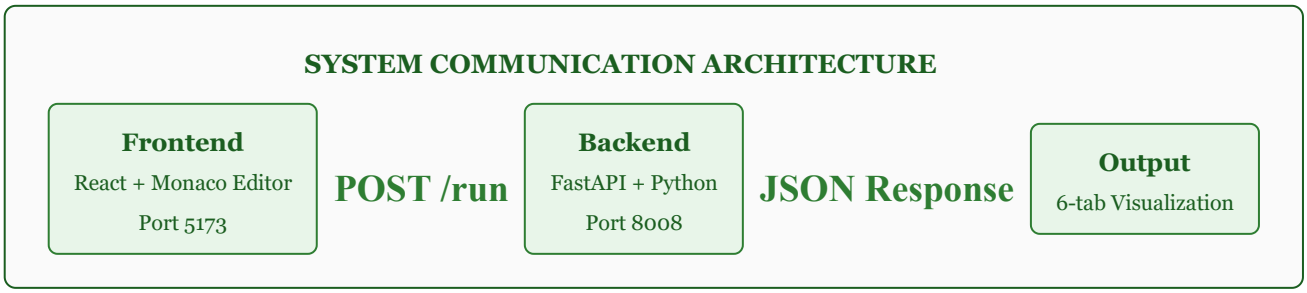


Figure 1: System communication architecture

Backend - Python 3.10+ with FastAPI 0.115+ on port 8008. Stateless POST /run endpoint.

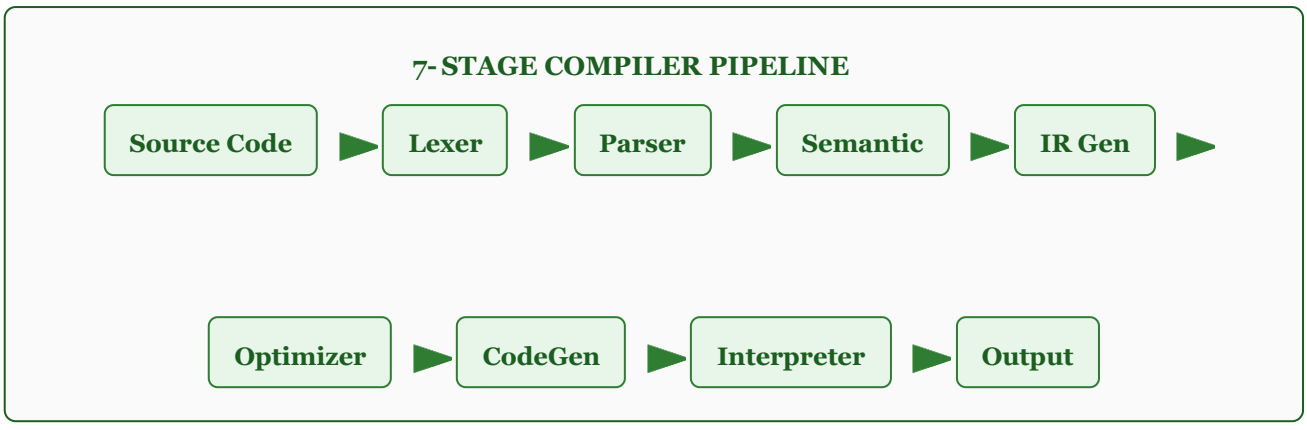
Frontend - React 19 with TypeScript 6 on port 5173 via Vite. Monaco-based IDE.

Communication: **POST http://localhost:8008/run** with JSON body { "code": "" }. The backend returns a single comprehensive JSON response. Fully stateless - no database, no sessions.

3.3 Compiler Pipeline - 7 Stages

The compiler follows a classic multi-stage pipeline. Each stage transforms the input from one representation to the next, with every stage's output visible in the Web IDE.

3.2 Compiler Pipeline



Stage 1: Lexical Analysis (Lexer)

File: backend/compiler/lexer.py

Converts raw source text into a flat stream of tokens. Recognizes 14 token types. Handles multi-character operators (>=, <=, ==, !=) with look-ahead. Skips whitespace and # comments. Produces Urdu error messages for malformed tokens.

Stage 2: Parsing (Parser)

File: backend/compiler/parser.py

Recursive-descent parser building an Abstract Syntax Tree (AST) from the token stream. Implements operator precedence: or -> and -> comparison -> add/sub -> mul/div -> unary -> primary. Produces 18 AST node types. "Did you mean?" suggestions via Levenshtein edit distance <=2.

Stage 3: Semantic Analysis

File: backend/compiler/semantic.py

Static analysis pass validating the AST. Type checking, scope resolution (block scoping with parent chain), undefined variable detection, static division-by-zero detection, function validation. Returns scoped symbol table with names, types, scopes, and depth.

Stage 4: IR Generation (TAC)

File: backend/compiler/ir_generator.py

Converts AST into Three Address Code (TAC). Uses temporaries (t0, t1...) and labels (L0, L1...). Instruction forms: assign, binary op, print, label, conditional jump, goto, return, call, array ops.

Stage 5: Optimization

File: backend/compiler/optimizer.py

Three optimization passes: (1) Constant Propagation, (2) Constant Folding (compile-time eval), (3) Dead Code Elimination. Folding runs twice to catch constants revealed by propagation.

Stage 6: Python Code Generation

File: backend/compiler/codegen.py

Translates AST into valid Python. Maps Urdu keywords: aur->and, ya->or, sahi->True, ghalat->False. Generates proper 4-space indentation. Supports if/else, while, def, return, print, arrays, calls.

Stage 7: Interpretation

File: backend/compiler/interpreter.py

Tree-walk interpreter executing the AST directly. Environment-based scoping with parent links. Function call stack with bound parameters. Safety: 10K iteration cap, 100 call-depth limit. Accepts pre-supplied input strings for API mode.

3.4 Technology Stack

Backend

Component	Technology	Version
Language	Python	3.10+
Web Framework	FastAPI	0.115+
ASGI Server	Uvicorn	0.32+
Validation	Pydantic	2.0+

Frontend

Component	Technology	Version
Language	TypeScript	6.0+
UI Framework	React	19.2+
Build Tool	Vite	8.0+
Code Editor	Monaco Editor	4.7+

3.5 System Architecture

The system is organized as a monorepo with clear separation of concerns:

Aspect	Details
Architecture	Monorepo - single repository with backend/ and frontend/
Communication	REST API (POST /run) with JSON request/response
Backend Port	8008 (FastAPI + Uvicorn)
Frontend Port	5173 (Vite dev server)
CORS	Enabled for all origins (development mode)
State	Fully stateless - no database, no sessions
Compiler Tools	Zero external - entirely hand-written in pure Python

3.6 Project Structure

The system is organized as a monorepo with clear separation of concerns:

```
Mini-Programming-Language/
+-- backend/
|   +-- compiler/              # Core compiler package
|   |   +-- lexer.py           # Stage 1: Tokenizer (~270 lines)
|   |   +-- parser.py          # Stage 2: Parser to AST (~545 lines)
|   |   +-- semantic.py        # Stage 3: Semantic analysis (~385 lines)
|   |   +-- ir_generator.py    # Stage 4: TAC generation (~220 lines)
|   |   +-- optimizer.py       # Stage 5: Optimization (~275 lines)
|   |   +-- codegen.py         # Stage 6: Python codegen (~180 lines) |
|   +-- interpreter.py     # Stage 7: Tree-walk interp (~375 lines)
|   |   +-- pretty_printer.py  # Terminal formatting
|   +-- main.py                # FastAPI app with POST /run |
+-- test_pipeline.py       # 12 integration tests
|   +-- requirements.txt
+-- frontend/
|   +-- src/
|   |   +-- api/compiler.ts    # REST API client |   |
+-- types/compiler.ts  # TypeScript interfaces
|   |   +-- components/
|   |   |   +-- Editor.tsx     # Monaco code editor
|   |   |   +-- OutputPanel.tsx
|   |   |   +-- SemanticPanel.tsx
|   |   |   +-- TACPanel.tsx
|   |   |   +-- PythonPanel.tsx
|   |   |   +-- ExamplesPanel.tsx (11 examples)
|   |   |   +-- Toolbar.tsx
|   |   +-- App.tsx
|   |   +-- main.tsx
|   +-- visuals/               # 11 screenshot PNGs
|   +-- package.json
|   +-- vite.config.ts
+-- .github/                   # Issue/PR templates
+-- LICENSE (MIT) / README.md / documentation.md / lang_doc.md
```

3.7 Setup & Deployment

Prerequisites: Python 3.10+ and Node.js 18+

Backend:

```
cd backend python -m venv venv
venv\Scripts\activate # Windows # source
venv/bin/activate # macOS / Linux pip
install -r requirements.txt
uvicorn main:app --reload --port 8008
```

Frontend:

```
cd frontend npm install npm run dev #
Open http://localhost:5173 in browser
```

API available at **http://localhost:8008** | IDE at **http://localhost:5173**

4. Results / Output

The system produces a rich set of outputs for every program execution. All outputs are simultaneously available in the web IDE's tabbed panel and in the JSON API response.

4.1 Web IDE Overview

The complete Web IDE with Monaco Editor, examples sidebar, and tabbed output panels:

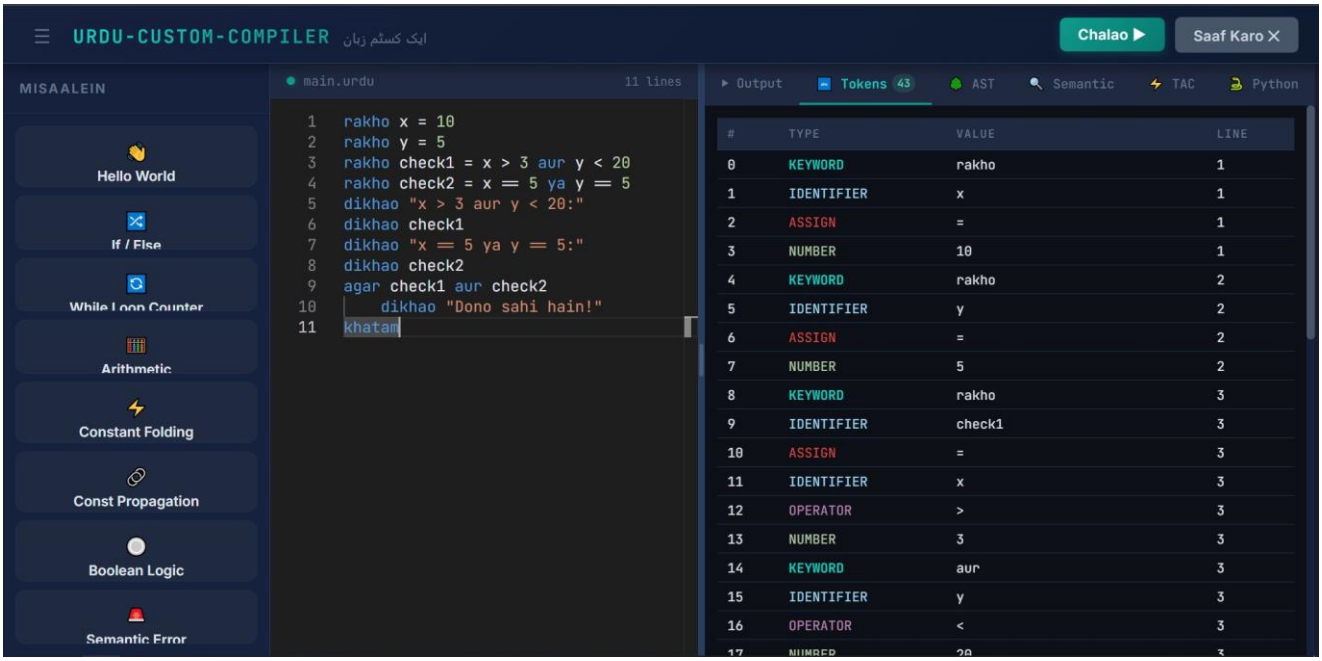
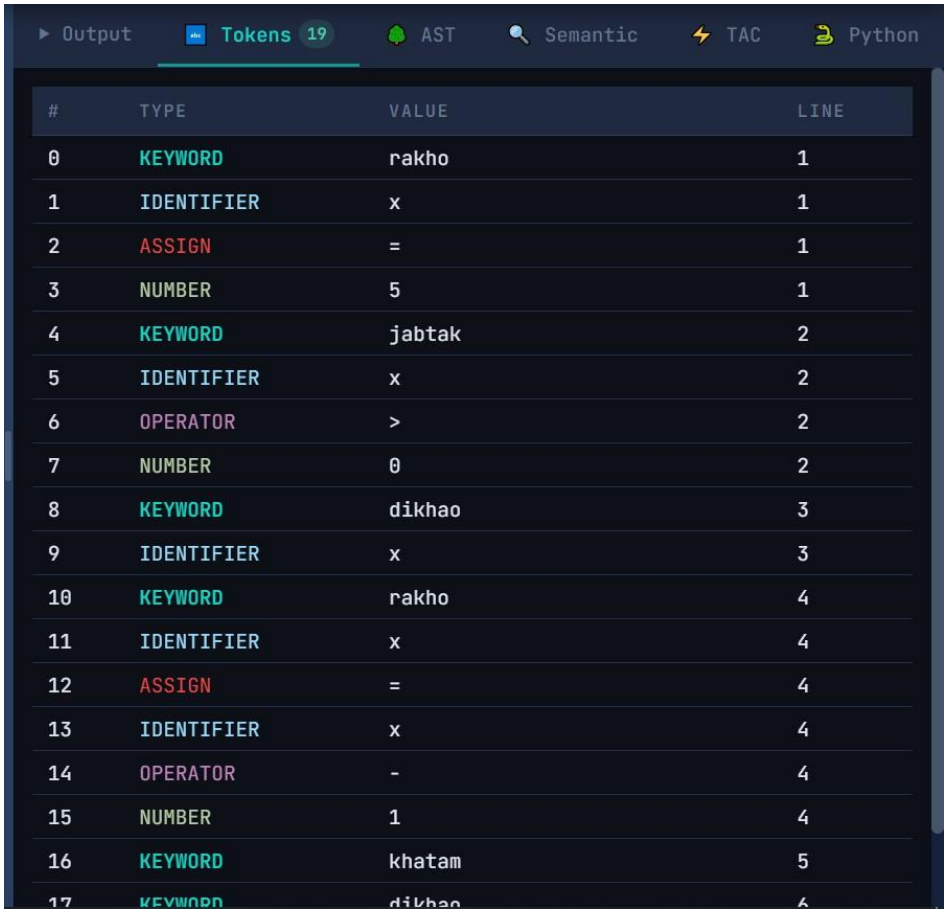


Figure 1: Complete Web IDE - Monaco Editor, examples sidebar, and tabbed output panels

4.2 Stage 1: Lexer - Tokenization



#	TYPE	VALUE	LINE
0	KEYWORD	rakho	1
1	IDENTIFIER	x	1
2	ASSIGN	=	1
3	NUMBER	5	1
4	KEYWORD	jabtak	2
5	IDENTIFIER	x	2
6	OPERATOR	>	2
7	NUMBER	0	2
8	KEYWORD	dikhao	3
9	IDENTIFIER	x	3
10	KEYWORD	rakho	4
11	IDENTIFIER	x	4
12	ASSIGN	=	4
13	IDENTIFIER	x	4
14	OPERATOR	-	4
15	NUMBER	1	4
16	KEYWORD	khatam	5
17	KEYWORD	dikhao	6

Figure 2: Token stream - each word, operator, and literal classified

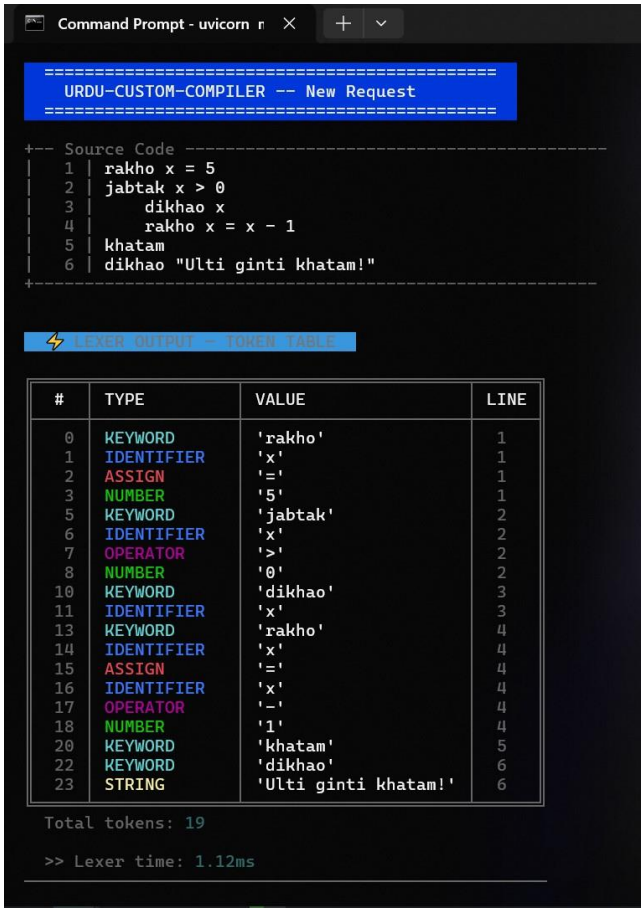


Figure 3: Web IDE Lexer panel showing full token list

4.3 Stage 2: Parser - Abstract Syntax Tree

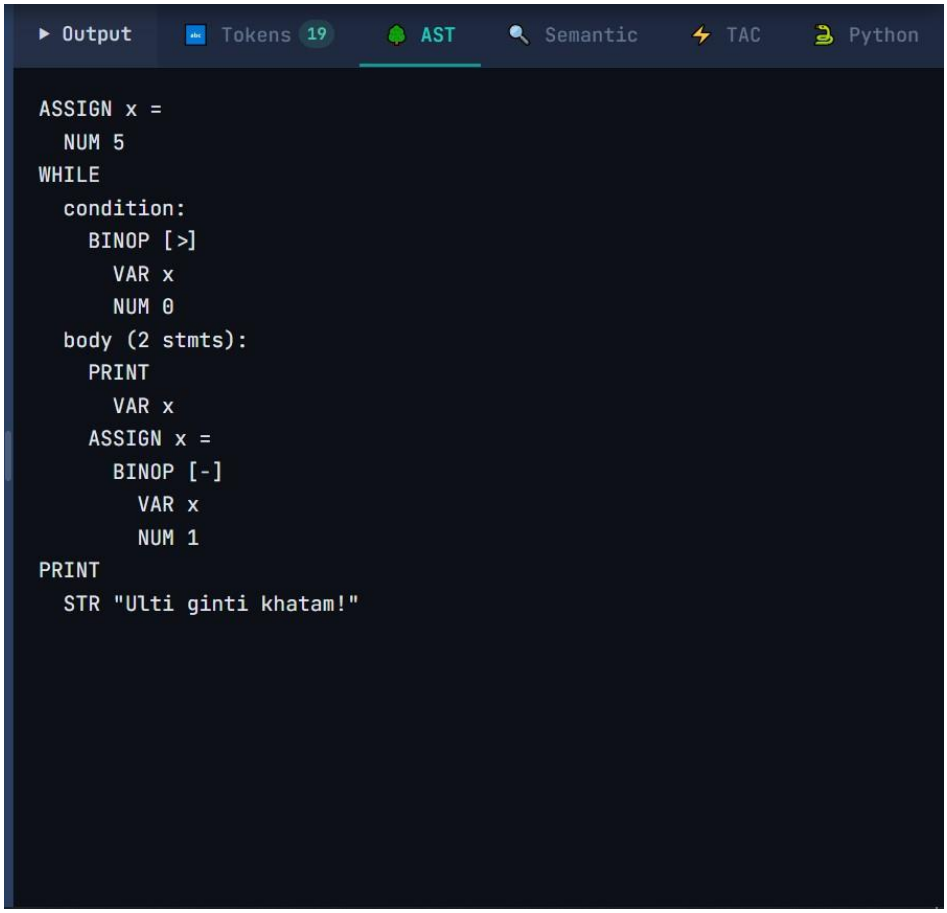


Figure 4: Abstract Syntax Tree showing program structure

Figure 5: Web IDE Parser panel with AST tree

4.4 Stage 3: Semantic Analysis

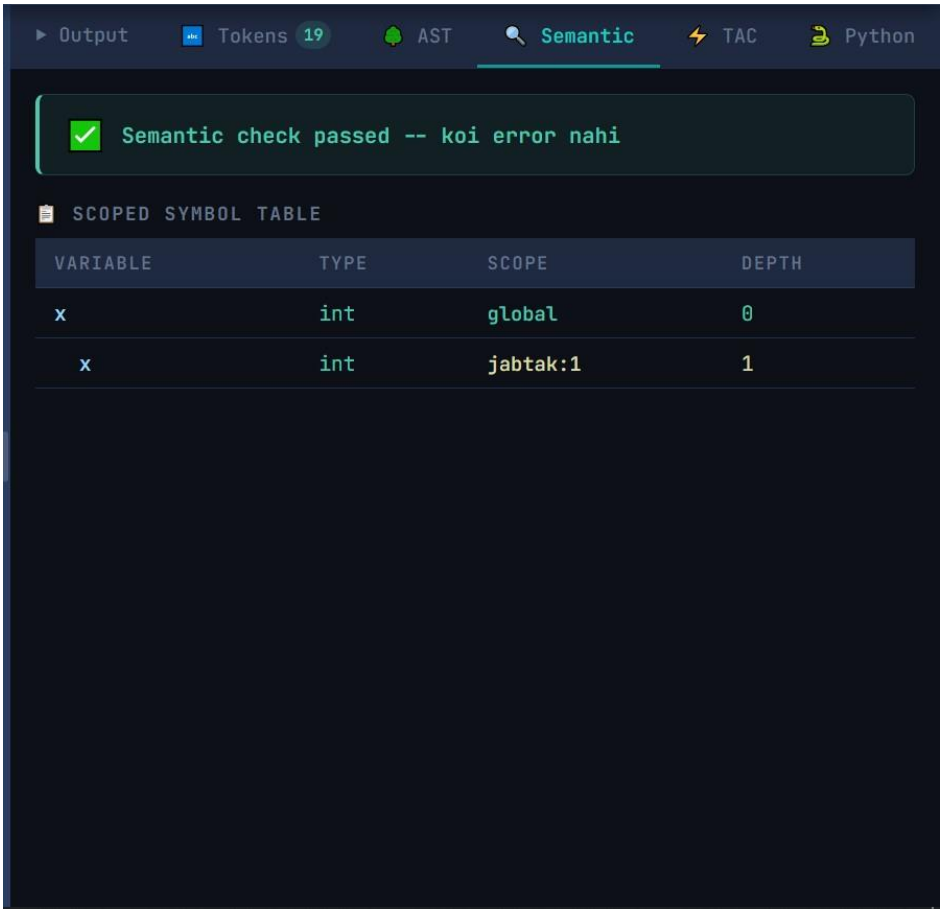


Figure 6: Symbol table with types, scopes, errors, and warnings

4.5 Stage 4 & 5: TAC Generation & Optimization

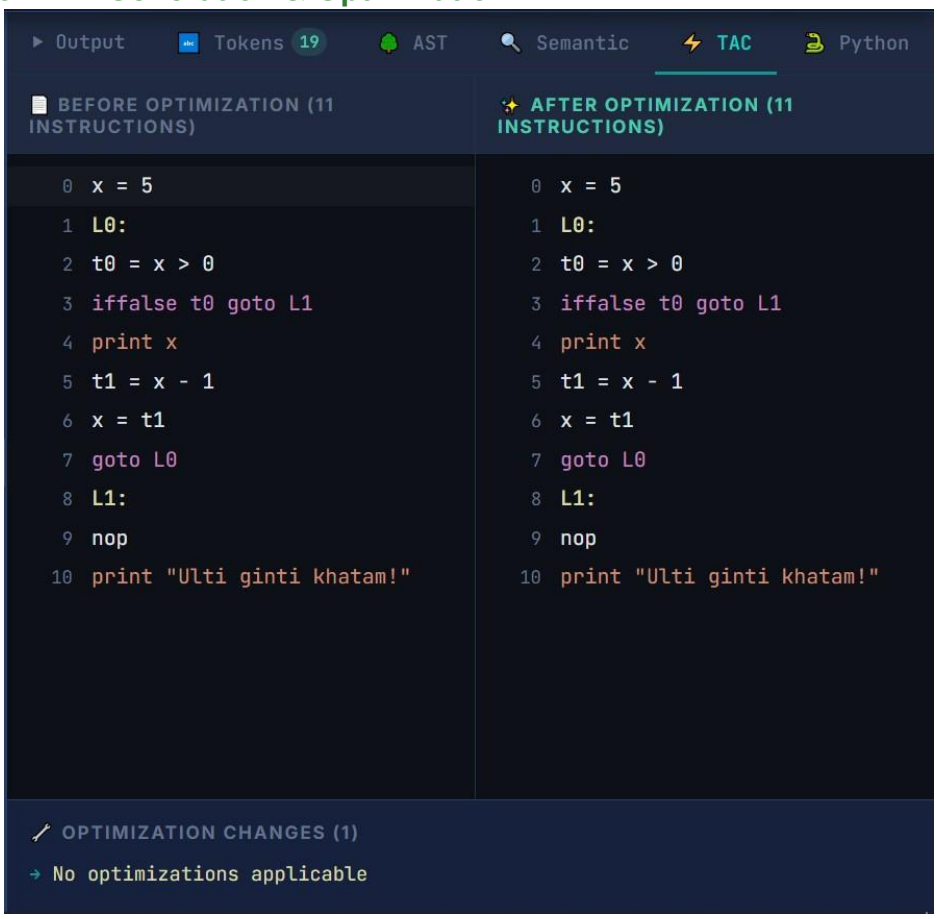


Figure 7: TAC instructions - original vs. optimized

Figure 8: Web IDE - TAC, optimization diff, and generated Python

4.6 Stage 6: Python Code Generation

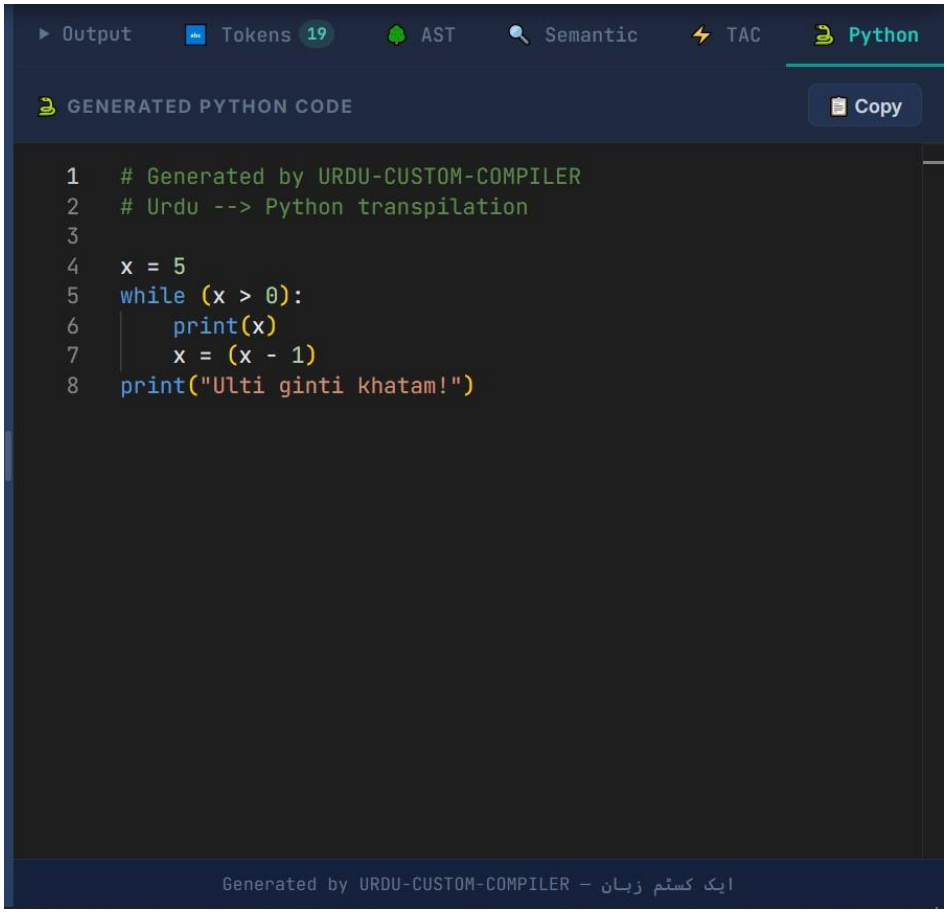


Figure 9: Generated Python - Urdu program translated to executable Python

4.7 Stage 7: Program Execution

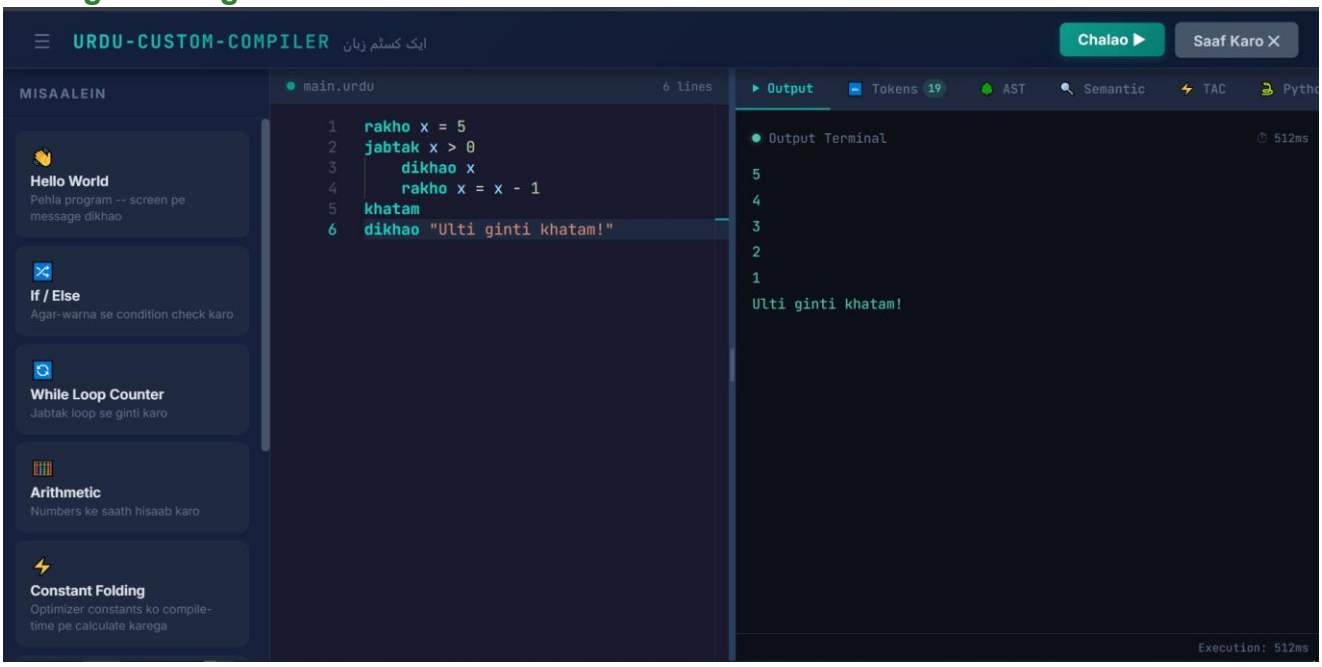


Figure 10: Final output panel showing program execution result

4.8 Example Programs

The IDE includes **11 built-in example programs** demonstrating every language feature:

#	Example	Features Demonstrated
1	Hello World	Basic printing
2	If / Else	Conditional branching with agar/warna
3	While Loop Counter	Loops with jabtak/khatam
4	Arithmetic	All arithmetic operators
5	Constant Folding	Compile-time constant evaluation
6	Constant Propagation	Variable-to-constant substitution
7	Boolean Logic	aur, ya, sahi, ghalat
8	Semantic Error	Error detection demonstration
9	Functions	banao, wapis, karo - definition, return, call
10	Arrays	Array creation, access, index assignment
11	User Input	input(), int(), str() - type casting with input

4.9 API Response Structure

POST /run returns a comprehensive JSON response:


```
{
"output": "42",
"tokens": [
{"index": 0, "type": "KEYWORD", "value": "rakho", "line": 1},
{"index": 1, "type": "IDENTIFIER", "value": "x", "line": 1},
{"index": 2, "type": "ASSIGN", "value": "=", "line": 1},
{"index": 3, "type": "NUMBER", "value": "42", "line": 1}
], "ast": "ASSIGN x =\n NUM
42",
"semantic": {
"errors": [], "warnings": [],
"symbol_table": {"x": "int"}, "scoped_symbols": [{"name": "x",
"var_type": "int", "scope": "global"}]
},
"tac": {
"original": ["x = 42"], "optimized": ["x = 42"],
"changes": ["No optimizations applicable"]
}, "generated_python": "x =
42\nprint(x)"
}
```

4.10 Performance Metrics

Typical timings for a medium-complexity program (20-30 lines):

Pipeline Stage	Typical Time
Lexer	~0.5 ms
Parser	~0.3 ms
Semantic	~0.2 ms
IR Gen	~0.2 ms
Optimizer	~0.3 ms
CodeGen	~0.1 ms
Interpreter	~0.5 ms
Total	**~2-5 ms**

4.11 Error Reporting

If any stage detects an error, the pipeline halts immediately and returns a structured error response with Roman Urdu message, line number, and error markers applied to the Monaco Editor (red underlines). For typos, Levenshtein-based suggestions are provided, e.g., "Kya aap 'khatam' likhna chahte the?" when the user types 'khaam'.

4.12 Test Coverage

test_pipeline.py contains **12 integration tests** running the full 7-stage pipeline:

Test	What It Verifies
Block Scoping	Variables inside agar/jabtak don't leak to global scope
Constant Folding	5 + 3 becomes 8 at compile time
Constant Propagation	Single-assignment vars replaced with literals
Dead Code Elimination	Unused temporaries removed from TAC
Semantic Error	Undeclared variable caught before execution
While Loop Scoping	Loop variable scoped correctly
Function (no params)	Function definition and call work correctly
Function (with params)	Parameter binding and return values
Array Operations	Array creation, access, and mutation
Nested Conditionals	agar within agar parses and executes correctly

Input Handling	Pre-supplied input strings work in API mode
Full Pipeline	End-to-end: source -> tokens -> AST -> TAC -> output

5. Conclusion

The Mini Programming Language project successfully demonstrates the complete design and implementation of a custom programming language with Roman Urdu syntax. From the ground up, we built:

- **A custom programming language** with 16 Urdu keywords supporting variables, arithmetic, control flow (if/else, while), functions with recursion, arrays, user input, type casting, and block scoping
- **A full 7-stage compiler pipeline** - entirely hand-written in Python with zero external tools - that transforms Urdu source through lexing, parsing, semantic analysis, IR generation, optimization, Python code generation, and interpretation
- **A modern Web IDE** built with React and Monaco Editor that provides real-time visualization of every compiler stage, making it an effective educational tool

The project proves that programming languages do not have to be English-centric. By using Roman Urdu syntax the natural way Urdu speakers type on standard keyboards - we lower the barrier to programming education for millions of Urdu speakers.

Achievements

- Designed and implemented a complete 7-stage compiler pipeline from scratch, covering all CompilerConstruction Lab topics
- Built a fully functional Roman Urdu programming language with 16 keywords, 4 data types, functions, arrays,and interactive I/O
- Delivered a production-quality web IDE with Monaco Editor integration, syntax highlighting, inline error markers,and 11 built-in examples
- Implemented three compiler optimization passes: constant propagation, constant folding, and dead codeelimination
- Wrote 12 integration tests covering all major language features and pipeline interactions
- Created intelligent error handling with Levenshtein-based "did you mean?" suggestions for keyword typos

Limitations & Future Work

- **File I/O:** Reading/writing files. Planned: kholo (open), likho (write)
- **Error Handling:** Try/catch. Planned: koshish (try), khata (catch)
- **String Functions:** Built-in: length, upper, lower, split
- **2D Arrays:** Multi-dimensional array support
- **CLI Runner:** Running .urdu files from command line without web IDE
- **Advanced Opts:** Loop unrolling, strength reduction, common subexpression elimination • **Urdu Script:** Native Nastaliq script support via Unicode input

6. References

[1] FastAPI Documentation. (2024). FastAPI - Modern, fast web framework. https://fastapi.tiangolo.com/

[2] Microsoft Corporation. (2024). Monaco Editor - The code editor that powers VS Code.https://microsoft.github.io/monaco-editor/

[3] React Documentation. (2024). React - The library for web and native user interfaces. https://react.dev/

[4] Pydantic Documentation. (2024). Pydantic - Data validation using Python type annotations. https://docs.pydantic.dev/

[5] Python Software Foundation. (2024). Python dataclasses. https://docs.python.org/3/library/dataclasses.html

[6] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.

[7] Vite Documentation. (2024). Vite - Next Generation Frontend Tooling. https://vitejs.dev/

[8] H0NEYP0T-466. (2026). URDU-CUSTOM-COMPILER / Mini Programming Language. GitHub. https://github.com/H0NEYP0T-466/URDU-CUSTOM-COMPILER